

Prediction of Geoeffective CMEs Using SOHO and SDO Observations

Prediction Workflow

Zhaoxin Yan

New Jersey Institute of Technology

1. Introduction

The fusion model combines SOHO/LASCO and SDO/AIA and HMI observations to predict CME geoeffectiveness. This workflow loads the pretrained model, predicts the test events, and reproduces the prediction results.

2. Prediction Workflow

2.1 Open the Notebook

Open `prediction.ipynb` in Google Colab using the *Open in Colab* button on the GitHub page. Select *Runtime* $\rightarrow$ *Run all*. The notebook installs dependencies and downloads the model weights and testing data. It uses a GPU when available and otherwise runs on the CPU.

2.2 Load the Model and Test Inputs

The checkpoint is `assets/dnet.best.pt`. The test-event list is `test.csv`, and prepared LASCO, AIA 193 Å, AIA 211 Å, and HMI inputs are supplied in `assets/test.tensors.zip`. The notebook verifies the downloaded files and extracts the inputs before prediction.

2.3 Run Prediction and Reproduce Results

The following notebook calls load the checkpoint, compute event probabilities, and display the prediction results:

```
ids, labels, probabilities = rf.predict(
    ckpt, tensor_dir, WORK / "test.csv", device)
counts = rf.confusion_counts(labels, probabilities)
figure = rf.plot_results(counts, WORK / "confusion_matrix.png")
```

For local execution, extract `DeepGeoCME.zip`, open a terminal in `DeepGeoCME`, and run:

```
pip install -r requirements.txt
python test.py
```

3. Prediction Output

The notebook displays the results and saves `confusion_matrix.png` and `predictions.csv`. Event dates and predicted probabilities are printed during prediction.

TN 14	FP 2
FN 2	TP 15

Results reproduced from the test-event predictions.
Rows: actual non-geoeffective and geoeffective classes;
columns: predicted non-geoeffective and geoeffective classes.